

Rust direct-rs ☞ C++ FFI Handshake (reference)

Andrew Jensen

2026-08-21 2026-08-24

1 Purpose

Reference for replacing the C++ DIRECT core (DirectOptimizer) with the Rust implementation in rust/direct-rs, keeping the C++ coordinator (OptimizerManager, CostFunctionManager, GPU cost) intact. This is a **handshake spec** — the concrete FFI boundary Rust and C++ must agree on — not a full implementation plan. It records every option considered so the reader can choose deliberately.

1.1 How to use this doc (learning path)

This is a living handshake **and** a tutor. Every section that teaches a Rust concept is tagged with three pointers:

- **Brown Book** — the free, interactive, visual book you prefer, at <https://rust-book.cs.brown.edu/>. A section like ch 04-02 opens at <https://rust-book.cs.brown.edu/ch04-02-references-and-borrowing.html>. The Brown variant adds quizzes and explainers the static book lacks.
- **rust-skills** — one of the 265 prioritized idioms at [../.pi/agent/skills/rust-skills/SKILL.md](https://github.com/andrewjensen/pi-agent/blob/main/.pi/agent/skills/rust-skills/SKILL.md) (rule ids like own-borrow-over-clone, one .md per rule).
- **normie** ☞ **idiomatic** — the shape a C++/Python developer reaches for first, and the idiomatic Rust you actually want.

The through-line question for every block: “**what did the Rust type system just make easier for me, and what did it make impossible?**” If a section reads as “translate C++ into Rust”, that’s the normie path — re-read it for the place where ownership/borrowing, or exhaustive =match=/enums, or iterator pipelines change what “good” means.

Carry the core line from the Brown book (its ch 4):

> Rust’s central feature is **ownership**, and it pervades even tiny examples.

Coming from **Python**: ch 3-8 is mostly review (variables, functions, structs, control flow). Camp on ch 4 (ownership), ch 9 (errors), ch 10 (traits/generics), ch 13 (closures/iterators), ch 15 (smart pointers), ch 17 (trait objects), ch 19 (unsafe).

Coming from **C++**: you’ll feel at home in ch 15 (Box ≈ unique_ptr) and ch 19 (unsafe ≈ raw pointers), and least at home in ch 4 (borrowing) — Rust’s borrow checker is stricter than =const&- discipline.

Suggested first pass (each 10-25 min, do the quizzes):

1. ch 04 — ownership/borrow/slices; the whole doc adheres to this.
2. ch 09-02 — Result + ?; the error-channel section arises from it.
3. ch 10-02 — traits; the Cost trait and IOptimizer options build on it.
4. ch 13 — closures/iterators; batching and POH gathering lean on it.
5. ch 15-01 — Box; where rust::Box<DirectOptimizer> comes from.
6. ch 19-01 — unsafe; what unsafe extern actually gives up.

2 Toolchain first — make the compiler your tutor

You cannot learn Rust by reading. The borrow checker is the teacher and it only speaks when you compile. Before Milestone 0, build a loop where you can ask the compiler a question and get an answer in under three seconds. Everything below this section assumes that loop exists.

2.1 The crate layout this doc implies

```

1 rust/direct-rs/
2   Cargo.toml
3   build.rs          # cxx-build: compiles the generated C++ glue
4   src/
5     lib.rs          # #[cxx::bridge] ONLY — the boundary, nothing else
6     pose.rs         # Pose, Direction                      (Steps 1-2)
7     hyperbox.rs     # Hyperbox, size(), trisect             (Steps 3, 7)
8     tree.rs         # BoxTree: two-level BTreeMap, gather, reinsert (Steps 4-5)
9     poh.rs          # convex_hull — pure                    (Step 6)
10    cost.rs          # trait Cost + AnalyticCost stub + CppCost impl (M6)
11    error.rs         # OptimizerError (thiserror)           (M8)
12    optimizer.rs     # DirectOptimizer::run — the loop       (M9)
13  tests/
14    parity.rs        # Tier-1 analytic golden vs C++        (Appendix F)

```

Why split it this way: `lib.rs` holding **only** the bridge is FFI containment — the unsafe surface is one file you can audit. Every other module is ordinary safe Rust, unit-testable with no C++ and no GPU in the room. **If a module can't be tested headlessly, FFI has leaked into it.** That single rule is worth more than any diagram in this doc.

2.2 Cargo.toml

```

1  [package]
2  name = "direct-rs"
3  version = "0.1.0"
4  edition = "2021"
5
6  [lib]
7  crate-type = ["staticlib", "rlib"] # staticlib for CMake to link; rlib for Rust tests
8
9  [dependencies]
10 cxx = "1.0"
11 ordered_float = "4"
12 thiserror = "2"
13
14 [build-dependencies]
15 cxx-build = "1.0"
16
17 [dev-dependencies]
18 approx = "0.5" # assert_relative_eq! for float parity
19 proptest = "1" # property tests for hull + trisect invariants

```

Note `[dev-dependencies]`: these compile for `cargo test` only and never ship into the C++ binary. That separation is why a test stub `cost (AnalyticCost)` costs the production build nothing.

2.3 build.rs (the other half of the handshake)

```

1 fn main() {
2     cxx_build::bridge("src/lib.rs")
3         .include("../include")           // where domain/cost.h lives
4         .flag_if_supported("-std=c++17")
5         .compile("direct_rs");
6     println!("cargo:rerun-if-changed=src/lib.rs");
7     println!("cargo:rerun-if-changed=../include/domain/cost.h");
8 }

```

The header C++ will `#include` lands at:

```

1 target/<profile>/cxxbridge/direct-rs/src/lib.rs.h

```

Open that file on day one. It is the ground truth for every “verify the verbatim signature” warning in this doc — `rust::Box<T>`, `rust::Slice<const T>`, the generated `Pose` struct, all of it. You never have to guess.

2.4 The six commands that are your feedback loop

Command	When / why
<code>cargo check</code>	every 30 seconds — type-checks without codegen; fastest tutor
<code>cargo clippy -- -W clippy::pedantic</code>	after it compiles — clippy names the idiom you missed
<code>cargo test</code>	after each Step; <code>cargo test hull</code> filters by name
<code>cargo expand --lib ffi</code>	to see what <code>#[cxx::bridge]</code> actually generated
<code>cargo doc --open</code>	read <code>ordered_float</code> / <code>cxx</code> docs offline, versioned to you
<code>cargo fmt</code>	never argue about layout; the formatter is canonical

Also bind: `rustc --explain E0502` (full prose for any error code) and `rustup doc --book` (the offline book, when you want the non-Brown text).

A note on clippy for a learner: run it with `-W clippy::pedantic` even though it’s noisy. Half of the 265 `rust-skills` rules are things clippy will tell you for free, with a link. Treat every clippy lint as a flashcard: read the name (`needless_range_loop` → `perf-iter-over-index`), fix it, move on.

2.5 Read the error, not the code

A `rustc` error has four parts: the code (E0502), the one-line claim, the annotated span, and a `help:` line that is usually the literal fix. Beginners stare at the span. The fastest learners read the `help:` first, then `rustc --explain`, and only then look at their own code. Appendix A’s table and the glossary’s error lookup are the project-specific version of that habit.

3 Learning Roadmap (plan it, then fill it)

This doc is big enough that the order you read it is a learning path. Use this roadmap as your map; the section headings are the terrain. Each milestone ends with a self-check you should be able to answer without looking before moving on.

3.1 Milestone 0 — why this is a two-language project

Goal: before any Rust, understand the split. The C++ side owns the cost and lifecycle; Rust owns the search algorithm. Everything else is plumbing between two runtimes.

- Read: Purpose, Contract being filled, One-picture flow.
- **Book:** none — orientation. Return to it after every milestone; the handshake diagram is the anchor.
- Self-check: state which side owns (1) the cost function, (2) the optimizer's search loop, (3) the worker thread, (4) the UI progress, and (5) the cumulative budget.

3.2 Milestone 1 — value types: Pose + Hyperbox

Goal: model the data before any algorithm. Internalize Copy vs Clone vs move, and why small POD structs cross FFI by value.

- Concept: `#[derive(Copy, Clone)]` on small structs; what a value is in Rust.
- **Book:** ch 04-01 (moves), ch 05-01 (structs), ch 06 (enums for Direction).
- **rust-skills:** own-copy-small, own-move-large, type-enum-states.
- Self-check: explain why Pose is Copy but a Hyperbox whose center is a Pose stays Copy (small POD), vs one that wraps a Vec which isn't — i.e. what makes a type Copy-eligible and what Copy costs you.

3.3 Milestone 2 — the BTreeMap design

Goal: the central data-structure decision, and the most Rust-flavored one in the doc. Learn Ord-based containers and the OrderedFloat wrapper.

- Concept: f64 is not Ord; BTreeMap needs a total order; wrapping floats.
- **Book:** ch 08 (collections) + the `ordered_float` crate docs (the Brown book has no OrderedFloat section — this is where you learn to read a dependency's docs).
- **rust-skills:** coll-map-choice, coll-set-membership, num-float-compare.
- Self-check: why can't you put f64 directly as a BTreeMap key, and what is the minimum wrapper that fixes it?

3.4 Milestone 3 — the gather (read-only borrow)

The first actual pass over the tree. Internalize **read-only borrow** = a `&self` pass with no mutation and no clone.

- Concept: `iter()`, `first_key_value()`, projecting (size, cost) without touching the box.
- **Book:** ch 04-02 (borrowing), ch 13-02 (iterator adapters).
- **rust-skills:** own-borrow-over-clone, perf-iter-over-index.
- Self-check: why does the gather take `&self` (not `&mut`), and what would break in the type system if you tried to `.clone()` the box out?

3.5 Milestone 4 — POH selection (pure transform)

Goal: `convex_hull(&[PohPoint]) -> Vec<PohPoint>` as a pure function. Internalize value-in/value-out and the absence of side effects.

- Concept: mapping (size, cost) pairs through a hull algorithm; returning owned `Vec<PohPoint>`.
- **Book:** ch 10-02 (traits) + ch 13 (closures as data).
- **rust-skills:** `anti-vec-for-slice`, `perf-collect-once`.
- Self-check: if `convex_hull` is pure, what do you pass in and what comes out, and how do you know the tree wasn't touched?

3.6 Milestone 5 — trisection + the ownership dance

The crown jewel of the doc and the deepest borrowing lesson: `take = move`, `trisection = owned transform`, `reinsert = move back`.

- Concept: `pop_first()` returns owned; `&mut self` is exclusive.
- **Book:** ch 04 (ownership) — the payoff milestone; re-read it.
- **rust-skills:** `own-move-large`, `mem-take-replace`, `own-arc-shared`.
- Self-check: why must you `pop/remove` the winner OUT of the map before trisectioning it, and why can't you mutate a map-resident box in place?

3.7 Milestone 6 — cost injection + the trait

Goal: the `Cost` capability; `batch-vs-serial`; the one-FFI-hop idea.

- Concept: `trait Cost, impl Fn vs Box<dyn Cost>`.
- **Book:** ch 10-02 (traits), ch 17 (trait objects), ch 13 (closures).
- **rust-skills:** `trait-dyn-vs-generic`, `closure-fn-trait-bounds`.
- Self-check: why is `Cost` a trait and not a concrete struct field, and when would you prefer `impl Fn vs Box<dyn Cost>`?

3.8 Milestone 7 — the FFI boundary (cxx)

Goal: see your Rust reach into C++. Own `#[cxx::bridge]`, opaque types, shared structs/enums, the no-panic rule.

- Concept: `extern "Rust" vs extern "C++"`, `Box<DirectOptimizer>` on C++ side, shared enum `> wildcard = _ arm`.
- **Book:** ch 19 (unsafe/FFI), ch 16 (threads for Send/Sync).
- **rust-skills:** `unsafe-safety-comment`, `err-no-unwrap-prod`.
- Self-check: what two things can never cross a `cxx` boundary, and why does that force the `status-struct + ProgressSink` shapes?

3.9 Milestone 8 — errors + status (the gate)

Goal: the error story inside (`Result + =?`) vs at the edge (status struct).

- Concept: two-tier errors; `Result` in, status out; panic never crosses.
- **Book:** ch 09 (all of it).
- **rust-skills:** `err-result-over-panic`, `err-thiserror-lib`, `err-question-mark`.
- Self-check: converting `OptimizerError` to a `RunOutcome.ok` — what is lost and what is kept, and why is that acceptable at the FFI edge?

3.10 Milestone 9 — assemble the loop + A/B

The finish: wire gather → POH → trisect → reinsert into one `run()` behind the `OptimizerBackend` A/B switch.

- Concept: glue, ownership flow end-to-end, the ok gate.
- **Book:** ch 04 + ch 13 once more (full recap).
- Self-check: walk the ownership of ONE hyperbox from insertion to trisection to reinsertion, naming each borrow/move.

Return here after every milestone and re-check the handshake diagram: each one fills a corner of it. The roadmap is the planning; the sections that follow are the terrain.

3.11 Milestone exercises — write these, don't read them

Each milestone above has a self-check **question**; here is the matching **keystroke**. Do the exercise before you read the corresponding Step, then read the Step and see what you'd have done differently. Nothing here needs C++, a GPU, or the real cost function — all of it is cargo test-able in the crate.

#	Exercise (write it, compile it, test it)	Done when
E1	<code>Pose + Direction; a const DIRECTIONS: [Direction; 6]; Pose::shifted(dir, d)</code>	a test asserts a s
E2	<code>Hyperbox::size();</code> unit-test that one trisection on the min axis shrinks size	<code>assert!(chil</code>
E3	Deliberately try <code>BTreeMap<f64, Hyperbox></code> ; read the error; then fix with <code>OrderedFloat</code>	you can quote th
E4	<code>BoxTree::insert + gather_poh</code> ; build a 5-box tree by hand in a test	gather returns o
E5	Try to <code>.clone()</code> a box out of <code>gather_poh</code> and make it compile with <code>&self</code>	you can explain
E6	<code>convex_hull</code> on a hand-made 6-point set with a known hull	test passes AND
E7	<code>trisect(parent) -> (Hyperbox, [Hyperbox; 2]);</code> assert depth sum increased by 1	three children, p
E8	<code>trait Cost + AnalyticCost</code> (e.g. sphere function <code>sum(x_i^2)</code>)	<code>run()</code> finds the
E9	<code>OptimizerError</code> with <code>thiserror</code> ; make <code>run</code> return <code>Result</code> , then flatten to <code>RunOutcome</code>	<code>ok=false</code> for a
E10	Wire the loop; run against <code>AnalyticCost</code> under a 20k budget	<code>calls + offs</code>

E3 and E5 are the two that **look** like a waste of time and aren't: they are deliberate failures. The point of writing code that doesn't compile is that the error message is the lesson, and you will meet both errors again for real later (see Appendix A). Budget ~20 minutes each; the rest are 30-60.

A good rhythm: exercise → the matching Step in “Designing DIRECT from scratch” → the Brown Book section it cites → cargo clippy on what you wrote. Four passes over the same idea, each from a different angle.

4 Contract being filled

The C++ `DirectOptimizer::Run()` contract, from `include/domain/direct_optimizer.h`:

Method	Signature	Semantics
<code>Run()</code>	<code>bool Run()</code>	false=error; loop guard <code>(calls+offset)<budget</code> &&
<code>GetCostFunctionCalls()</code>	<code>unsigned int</code>	<code>cost_function_calls_ + call_offset_</code>
<code>GetNonFiniteCount()</code>	<code>unsigned int</code>	count of non-finite evals
<code>GetOptimumLocation()/Value()</code>	<code>Point6D / double</code>	denormalized physical best / best cost
<code>Stop()</code>	<code>void</code>	cooperative, checked between iterations
<code>SetCallOffset(u)</code>	<code>void</code>	cumulative offset; canonical caps trunk 20k → 2x branch 2
<code>SetIterationCallback(f)</code>	<code>void</code>	after each <code>ConvexHull+Trisect</code> , ~30fps
<code>SetImprovementCallback(f)</code>	<code>void</code>	on new best (emits <code>UpdateOptimum</code>)
<code>SetBatchCost(f)</code>	<code>void</code>	optional U11 batch sibling

Options struct (8 fields, all fail-fast unless default): `selection (SelectionMode=Original)`, `=epsilon=0.0`, `=delta_limit=bool(false)`, `=size_measure=SizeMeasure=L2`, `=split_rule=SplitRule=OneSide`, `=ties=TieSelection=All`, `=hidden_constraints=bool(false)`, `=globally_biased=bool(false)`.

Truths that must survive any backend:

- cumulative `calls+offset` budget semantics (canonical caps: trunk 20k → 2x branch 25k/30k → leaf 35k, per `direct_optimizer.h`)
- the SAME optimum for the same cost sequence (Tier-1 analytic golden is the judge)
- eval ordering: `calls++` → non-finite → optimum update → storage → improvement callback
- evaluate executed **serially** on one worker thread (GPUModel pose is mutated per call)

4.0.1 Rust learning — “options as data” vs the god object

The C++ Options struct is already **value-shaped** (a struct passed by value) — a good instinct. The normie-from-C++ error is to treat it as a bag of bools/enums read by `if` inside `Run()`. Rust nudges you to make the impossible state **unrepresentable** rather than merely checked.

- **Book:** ch 05 (structs), ch 06 (enums + exhaustive match), ch 10-02 (traits).
- **rust-skills:** `type-enum-states`, `type-newtype-validated`, `api-parse-dont-validate`, `api-common-traits`, `num-float-compare`, `anti-over-abstraction`.

Normie (a loose struct with raw codes; callers must remember valid values):

```
1 pub struct Options { pub selection: u8, pub epsilon: f64, pub delta_limit: bool }
2 // a caller writing selection: 8 compiles fine and is wrong at runtime
```

Idiomatic (a named enum + derive so invalid values can't exist):

```
1 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
2 pub enum SelectionMode { Original } // only one legal value today
3 #[derive(Debug, Clone, Copy, Default)]
4 pub struct Options { pub selection: SelectionMode } // Default == the C++ default
```

Why `epsilon`, `size_measure`, `split_rule`, `ties`, `hidden_constraints`, `globally_biased` are **absent** above: the C++ side fails-fast on any non-default value. The idiomatic Rust is to **not declare the knob until you implement it** (anti-over-abstraction). Declaring a field you cannot honor is a promise the type system can't keep; dropping it entirely is a compile-time guarantee that beats the run-time `ValidateOptions()`.

The cumulative budget caps (trunk 20k → 2x branch 25k/30k → leaf 35k) are better as **data** than scattered literals — a single `Budget` newtype (`type-newtype-ids`) and the ladder defined once, used everywhere. If you ever **compute** a budget, reach for `num-overflow-explicit` (checked/saturating arithmetic) rather than letting a `+=` overflow wrap.

5 Designing DIRECT from scratch, step by step

This is the procedural heart. Forget the FFI and options for a moment: here we derive the Rust DIRECT data model and loop **one step at a time**, in the order you'd discover them while porting. Each step introduces exactly one idea, usually one type, and checks itself before moving on. This is where the “normie → idiomatic” comparisons get their teeth — each step is a real decision, not a style preference.

Throughout, “deduce” means **let the compiler + the borrow checker teach you**, not just copy a pattern. The Brown Book's ch 4 is the reference for steps 3-5; re-open it when a step feels abstract.

5.1 Step 1 — model a point: ‘Pose’ (value type)

The whole algorithm is a search over 6-D poses. First question: what is a pose **on its own**? A bundle of six `f64` whose identity is its contents. That is a value type — and in Rust the first reflex is to ask: should this be `Copy`?

Normie-from-C++ (a class with methods + hidden members):

```
1 | class Pose { double x,y,z,xa,ya,za; /* getters, setters... */ };
```

Idiomatic (a plain value struct, `Copy`, fields public):

```
1 | #[derive(Clone, Copy, PartialEq, PartialOrd)]
2 | pub struct Pose { pub x:f64, pub y:f64, pub z:f64, pub xa:f64, pub ya:f64, pub za:f64 }
```

Why: `Pose` is `6×f64=` 48 bytes — genuinely small — and its equality/order are meaningful (`PartialEq`, `PartialOrd`). `Copy` is right for a small plain-data value (rust-skills `own-copy-small`): passing it around copies 48 bytes (cheap), and `Copy` lets you read it out of a map key without `.clone()` ceremony.

- **rust-skills:** `own-copy-small`, `type-enum-states`.
- **Book:** ch 04-01 (`Copy` vs `move`), ch 05-01 (structs).

Stop. **Check** — is a `Pose` a thing you own or a thing you copy? It is both — `Copy` is the honest label for “duplicating it is cheap and preserves meaning,” which tells the compiler it may copy at will.

5.2 Step 2 — model an axis: ‘Direction’ (enum, not ints)

DIRECT trisects along one of six axes. The normie input is ints 0..5 or named constants; Rust has a better tool: `Direction` as an enum.

```
1 #[derive(Clone, Copy)] // tiny tag — keep it Copy
2 pub enum Direction { X_DIR, Y_DIR, Z_DIR, XA_DIR, YA_DIR, ZA_DIR }
```

An enum buys three things the normie approaches lack:

1. no invalid value (no “axis 7”) — `type-enum-states`;
2. exhaustive match — the compiler forces all six cases; and
3. it pairs with an index into depths: `[u32; 6]` via a small `const` mapping array instead of a cast. You already have ‘`const DIRECTIONS: [Direction; 6]`’.
4. **rust-skills:** `type-enum-states`, `pat-exhaustive-enum`.
5. **Book:** ch 06 (enums + match).

5.3 Step 3 — model a box: ‘Hyperbox’

Now the atom of the search. A hyperbox has a center (`Pose`), a per-axis depth depths: `[u32; 6]` (how many times each axis has been trisected), and a cost at that center (`Option<f64>` — we’ll see why).

Normie-from-C++ (a class with a `size()` method + friends):

```
1 class HyperBox6D { /* ... */ double GetSides(); /* ... */ };
```

Idiomatic (a plain struct + a method on an `impl`):

```
1 #[derive(Clone, Copy)]
2 pub struct Hyperbox {
3     pub cost_at_center: Option<f64>,
4     pub center: Pose,
5     pub depths: [u32; 6],
6 }
7 impl Hyperbox {
8     /// L2 norm of box extent from depths — the "size" used by POH.
9     pub fn size(&self) -> f64 {
10         self.depths.iter()
11             .map(|e| 3f64.powf(-2.0 * (*e as f64)))
12             .sum::<f64>()
13             .sqrt()
14     }
15 }
```

Two Rust habits to internalize here:

- **`Option<f64>` for “not scored yet.”** The center box inherits its parent’s cost; the two shifted boxes don’t exist yet — pending a batch eval. `Option`

makes “not yet known” an explicit state instead of a magic `DBL_MAX=-1=` sentinel. This is `type-option-nullable + api-parse-dont-validate`: represent the absence; don’t smuggle a fake number.

- **A method computes what isn't stored.** `size()` is derived from `depths`

every time; you don't cache it in the struct. Fewer fields = fewer places to be inconsistent. (Add a cache only if it shows up in the profiler = `perf-profile-first`.)

- **rust-skills:** `type-option-nullable`, `api-parse-dont-validate`, `own-copy-small`, `perf-profile-first`.
- **Book:** ch 05-01 (structs), ch 06 (Option), ch 13-02 (iterators — the `sum` is already an iterator pipeline).

Stop. **Check** — why `Option<f64>` for `cost` rather than `f64` with `-1` “meaning missing”? Because `-1` might be a valid `cost` that gets silently used; `None` is a kind the compiler forces you to handle before you can use the box.

5.4 Step 4 — the two-level ‘BTreeMap’ (the Ord problem)

The heart of the data model is a container of hyperboxes keyed by `size` (outer) then `cost` (inner). The moment you reach for a sorted map you hit a very Rust-specific wall: **f64 is not Ord**.

Why: `BTreeMap` (and `BTreeSet`, `BinaryHeap`, `sort`) needs a **total order** — the `Ord` trait. But `f64` has `NaN`, which compares “not equal to everything, including itself,” breaking total-order. So the standard library deliberately refuses `f64` as a map key / in `Ord` contexts.

Normie-from-C++ (map on `std::map<double, ...>` — C++ lets you, `NaN` foot-gun later):

```
1 | std::map<double, std::vector<Box*>> columns; // NaN quietly poisons comparisons
```

Idiomatic (wrap the float + add an id for ties):

```
1 | use ordered_float::OrderedFloat; // newtype: f64 that IS Ord
2 | pub type SizeKey = OrderedFloat<f64>;
3 | pub type CostKey = (OrderedFloat<f64>, u64); // (cost, unique id) so equal costs don't collide
```

`OrderedFloat<f64>` is a newtype that implements `Ord` by asserting the value is never `NaN`. It's the idiomatic escape hatch when you know your floats are total (your sizes/costs are never `NaN`). The `u64` tiebreak makes the inner key injective.

- **rust-skills:** `num-float-compare`, `type-newtype-ids`, `coll-map-choice`.
- **Book:** the `ordered_float` crate isn't in the Brown book — this is your first “read a dependency's docs” moment. The concepts (traits, newtype, `Ord`) are ch 10-02 + ch 08.

Stop. **Check** — why is the inner key `(cost, u64)` rather than just `cost`? Because two boxes can have the same `cost`; a `BTreeMap` keyed by `cost` alone would let the second silently overwrite the first. The monotonically increasing `u64` box id breaks the tie so both survive — `type-newtype-ids` in a container.

5.5 Step 5 — the gather (read-only pass)

Now the first read over the tree: collect, for every size-row, the min-cost box's (size, cost) pair — the input to POH. This is a **read-only borrow** pass = &self, no mutation, no clones.

Normie-from-C++ (a const& method that iterates + copies minima out):

```
1 | std::vector<std::pair<double,double>> gather(const std::map<...>& rows) {...}
```

Idiomatic (read-only iter() + first_key_value(), projecting two numbers):

```
1 | fn gather_poh(&self) -> Vec<PohPoint> {
2 |     let mut rows = Vec::new();
3 |     for (size_key, cost_val) in self.bboxes.iter() {           // &, immutable
4 |         if let Some((key, _box)) = cost_val.first_key_value() { // &, immutable
5 |             rows.push(PohPoint {
6 |                 size: (*size_key).into_inner(), // copy the f64 out of the key
7 |                 cost: (key.0).into_inner(),     // copy the cost, not the box
8 |             });
9 |         }
10 |     }
11 |     rows
12 | }
```

The teaching gold is **what did NOT happen**: we never `.clone()` a Hyperbox, we never mutated the tree, so `gather_poh` is &self and it's callable while other borrows are live. If the normie version cloned boxes into a `Vec<Hyperbox>`, it'd be forced into `&mut self` (or fight the borrow) and carry the whole box when only two numbers were needed. The borrow checker **points to** the leaner shape.

- **rust-skills**: own-borrow-over-clone, perf-iter-over-index, own-slice-over-vec.
- **Book**: ch 04-02 (borrowing — the whole step), ch 13-02 (iterator adapters), ch 04-01 (Copy — why `*size_key` is a copy, not a move).

Stop. **Check** — why does the `gather` return `Vec<PohPoint>` (owned copies of (size, cost)) instead of `Vec<Hyperbox>` or borrows into the map? Because POH only needs two numbers; borrowing into the map would ripple `&mut self` rules; and owned small copies are cheap. Return *values*, not *references* into yourself — the whole point of value types.

5.6 Step 6 — POH selection (pure transform)

POH (potentially-optimal hyperboxes) — convex hull / pareto over the `PohPoint` list. This is a **pure function**: `convex_hull(&[PohPoint]) -> Vec<PohPoint>`. It touches no tree, borrows nothing lasting, allocates only its own return.

Normie-from-C++ (a method that pushes/pops a member list):

```
1 | void convexHull() { m_poh.push_back(...); m_poh.pop_back(); } // mutates THIS
```

Idiomatic (pure: takes a slice, returns an owned Vec):

```

1 fn convex_hull(poh: &[PohPoint]) -> Vec<PohPoint> { // &[T], not &Vec<T>
2     let mut hull: Vec<PohPoint> = Vec::new();
3     for p in poh.iter() {
4         while hull.len() >= 2
5             && cross(hull[hull.len()-2].size, hull[hull.len()-1].size, p.size) <= 0.0 {
6             hull.pop();
7         }
8         hull.push(*p);
9     }
10    hull
11 }

```

Why does it feel cleaner than the C++? No side channel — input is a slice of points, output is an owned Vec, nothing else changes. The absence of side effects is not a preference; it’s what lets you call it headlessly in a unit test with a stub cost (test-mock-traits) with no GPU or UI. (The `&[T]` habit = own-slice-over-vec also makes it FFI-ready.)

- **rust-skills:** anti-vec-for-slice, own-slice-over-vec, perf-collect-once, test-mock-traits.
- **Book:** ch 10-02 (pure funcs over traits), ch 13 (iterators), ch 04 (why an owned `Vec<PohPoint>` return is fine — it’s moved out, not copied out).

5.7 Step 7 — trisection + the ownership dance (THE lesson)

The step that ties the room together: take a selected box out, trisect it into center + two shifted boxes, reinsert them. This is where Rust’s ownership model forces a structurally correct design that C++ lets you violate.

Normie-from-C++ (mutate the box in place, resize a vector):

```

1 void trisect(HyperBox &b, std::vector<...> &out) { /* mutate b, push children */ }

```

Idiomatic (consume the parent, return the children):

```

1 fn trisect(mut parent: Hyperbox) -> (Hyperbox, PendingTwo) { // take = move
2     let (min_idx, _) = parent.depths.iter().enumerate()
3         .min_by_key(|&(_, v)| *v).expect("6-element array");
4     parent.depths[min_idx] += 1;
5     // center keeps parent's cost; two shifted children get cost_at_center: None
6     // ... (build pos/neg shifted boxes)
7     (parent, [pos, neg])
8 }

```

and the caller re-inserts:

```

1 let (center, [pos, neg]) = Hyperbox::trisect(winner); // winner moved OUT
2 // batch-eval pos/neg's new centers first, then reinsert with costs

```

Why does Rust force “take it out, transform, put it back”?

1. **The map key is the size.** `BTreeMap<SizeKey, ...>` — if you mutated a map-resident box’s `depths`, its `size()` changes, so its `SizeKey` would be wrong and the map corrupted. Rust’s borrow rules make this hard to even express.
2. **`pop_first()` returns *owned*.** On the inner map it gives you the winner as an owned `Hyperbox` — no lifetime to fight, free to move/transform. That’s mem-take-replace in the wild.

3. **Reinsert = move=.** `boxes.entry(new_key).or_default().insert(id, child)` moves the child in. The tree **owns** it now; your Rust code no longer does.

This is the single most important ownership lesson in the doc. Carry this: **the three phases map to three ownership states — borrow to gather, own to transform, move to store — and the borrow checker keeps them honest.**

- **rust-skills:** `own-move-large`, `mem-take-replace`, `own-copy-small`, `api-builder-must-use`.
- **Book:** ch 04 (all of it); ch 15 (why `Box` fits `unique_ptr`-shaped ownership); ch 08 (`BTreeMap` `entry/remove` API).

Stop. **Check** — why can't you just mutate a box inside the map and "put it back in place"? Because its size (the map key) changed, and Rust won't give you an `&mut` into a container while you're keyed by its value (the map re-balances on `insert` $\neq$ `remove`, not in place).

That is the whole algorithm core in seven steps. What remains — cost injection, batching, FFI, errors — is scaffolding around this loop, which is why the Roadmap routes you back to Steps 5-7 the most.

6 The concrete handshake

6.1 One-picture flow

```

1  C++ (coordinator)                                | Rust (direct-rs)
2  -----
3  RunDirectStage                                   |
4  backend = launch.optimizer_backend               |
5  if Rust:                                         |
6      CppCost h = build_cpp_cost(BuildGpuCostAdapter...) // sealed std::function
7      out = direct_rs::new_direct_optimizer(range,start,budget,mode)->run(*h)
8      cost_calls_=out.calls; opt_loc/val=out.best... | Box<DirectOptimizer>.run(&CppCost)
9  else: existing DirectOptimizer path              | ⚡
   ↳ RunOutcome{best,best_cost,calls,ok}

```

6.2 Rust bridge (direct-rs/src/lib.rs)

```

1  #[cxx::bridge(namespace = "direct_rs")]
2  pub mod ffi {
3      struct Pose { x: f64, y: f64, z: f64, xa: f64, ya: f64, za: f64 }
4
5      enum PohMode { ConvexHull = 0, Pareto = 1, Aggressive = 2 } // shared enum
6
7      unsafe extern "C++" {
8          include!("domain/cost.h");
9          type CppCost;
10         // cost hot path: 6 doubles by-value (bind to CppCost::evaluate6)
11         fn evaluate6(self: &CppCost, x: f64, y: f64, z: f64,
12                     xa: f64, ya: f64, za: f64) -> f64;
13     }
14
15     extern "Rust" {

```

```

16     type DirectOptimizer;
17     fn new_direct_optimizer(range: &Pose, start: &Pose,
18                           budget: u32, mode: PohMode) -> Box<DirectOptimizer>;
19     fn run(&mut self, cost: &CppCost) -> RunOutcome;      // M1: run-to-completion
20     // M2 (live progress): see "Milestone options" below
21 }
22
23 // shared struct declared INSIDE the bridge so cxx generates the C++ struct
24 struct RunOutcome { best: Pose, best_cost: f64, calls: u32, ok: bool }
25 }

```

6.2.1 Rust learning — what does “shared struct” mean, and why does cxx care?

The ‘struct Pose { x: f64, ... }’ inside ‘#[cxx::bridge]’ is a **shared struct** — one definition, two languages, byte-identical layout. cxx generates a C++ ‘struct Pose’ with the same fields in the same order, so a ‘Pose’ can be passed **by value** across the boundary with zero serialization.

- **Book:** ch 05-01 (defining a struct), ch 19 (unsafe/FFI — how a ‘#[repr]’ ties layout).
- **rust-skills:** api-non-exhaustive, name-acronym-word (name it ‘Pose’, not ‘POSITION6D’), own-copy-small. ‘Pose’ is $6 \times f64 = 48$ bytes — small and ‘Copy’-able, so passing it by value is cheap (normie-from-Python would over-use references/boxes worried about cost; here by-value is the fast choice).

Normie-from-C++ (reach for a smart pointer / reference everywhere):

```

1 | fn run(&mut self, cost: &CppCost, point: &Pose) -> f64    // borrow of a 48-byte POD

```

Idiomatic (small value types cross by value):

```

1 | fn evaluate6(&self, x: f64, y: f64, z: f64, xa: f64, ya: f64, za: f64) -> f64

```

Notice the doc chose the **six-double** scalar form for the hot path, not the ‘&Pose’. That’s a deliberate cxx/layout decision — a bare ‘Pose’ by value would also work, but six scalars make the ABI trivially C-compatible and dodge any implicit padding question. The takeaway: **at an FFI boundary, prefer the simplest ABI (plain doubles / POD struct) so both compilers agree on layout without you reasoning about padding.** Brown Book ch 19-04 explains why layout matters for FFI.

A shared ‘enum PohMode’ becomes a C++ ‘enum class PohMode’ on the other side. Because a C++ ‘enum class’ can legally hold an out-of-list value, every match on a shared enum in Rust must include a wildcard ‘_’ arm — cxx cannot prove C++ will only send a listed discriminant. This is a genuinely Rust-flavored ~normie=trap: in C++ you never match exhaustively; in Rust the compiler **forces** the ‘_’ anyway.

```

1 | let mode = match mode {
2 |     ffi::PohMode::ConvexHull => ...,
3 |     ffi::PohMode::Pareto    => ...,
4 |     ffi::PohMode::Aggressive => ...,
5 |     _ => Err(UnknownPohMode), // cxx shared enums can't be trusted to be in-list
6 | };

```

(**Book ch 06**, exhaustive matching; this is exactly why Rust prefers exhaustiveness.)

6.3 C++ cost-side change (include/domain/cost.h)

Existing opaque CppCost wraps `std::function<double(const Point6D&)>` and exposes `evaluate(const Point6D&)`. Add a 6-double bridge method so the cxx static assertion matches the Rust bridge:

```

1 class CppCost {
2     double evaluate(const Point6D& point) const { return fn_(point); }
3     // bridge: build Point6D from 6 doubles, delegate to fn_ (no behavior change)
4     double evaluate6(double x, double y, double z,
5                     double xa, double ya, double za) const {
6         return evaluate(Point6D(x, y, z, xa, ya, za));
7     }
8 };

```

6.4 C++ call site (A/B seam) — inside `OptimizerManager::RunDirectStage` construction

Scout finding: the clean kernel-level A/B point is the **DirectOptimizer construction inside `RunDirectStage`** (`optimizer_manager.cpp:1303`), NOT the whole-run `OptimizerRunDriver` (driver wraps the 3-stage manager; we are single-stage).

```

1 // settings/launch: enum class OptimizerBackend { Cpp, Rust };
2 if (launch_.optimizer_backend == OptimizerBackend::Rust) {
3     auto h = build_cpp_cost(BuildGpuCostAdapter(...)); // std::unique_ptr<CppCost>
4     auto box = direct_rs::new_direct_optimizer(range, start, budget, mode);
5     auto out = box->run(*h); // *h derefs unique_ptr → CppCost& (matches &CppCost)
6     cost_function_calls_ = out.calls;
7     current_optimum_location_ = Point6D(out.best.x, out.best.y, /* ... */);
8     current_optimum_value_ = out.best_cost;
9     if (!out.ok) { emit OptimizerError("rust direct failed"); error_occurred_ = true; }
10 } else {
11     // existing DirectOptimizer block, byte-identical
12 }

```

Rules:

- Do NOT put backend into `DirectOptimizer::Options` — `ValidateOptions` fail-fast any non-default field. A settings-service field + `OptimizerRunLaunch` is the clean home.
- The `=CppCost` handle must out-live the run call (it holds the sealed `std::function` capturing GPU state).

7 Options & landmarks

7.1 backend selection — A/B switch mechanics

1. **Runtime C++ virtual interface:** define an `IOptimizer` abstract seam that a wrapper around the existing C++ `DirectOptimizer` AND the Rust backend both implement; select at runtime (via the `OptimizerRunController` factory) without touching the stage loop. Cleanest long-term (one objective seam the whole app uses); more moving parts up front. cxx opaque types compose with C++ inheritance (move-only smart-ptr-handled structs).

2. **Compile/link-time toggle** (CMake): `option(JTML_USE_RUST_OPTIMIZER ...)`; conditionally `target_link_libraries(direct-rs-lib)` + define `JTML_USE_RUST_OPTIMIZER`. Pick at build; zero runtime divergence; requires rebuild to toggle.
3. **Direct single-branch swap** in `RunDirectStage` (the flow above): least injection; both implementations coexist behind `optimizer_backend`. Fastest to A/B; coordination layer still knows Rust directly.

Recommend for expedient A/B: **#3 first**, then promote to a runtime interface (#1) if you want a settings toggle without recompiling.

7.2 Milestones — run-to-completion vs live progress

- M1 (recommended first): `run(&mut self, cost) -> RunOutcome` — run to completion, no live UI progress. Shortest to a compilable A/B.
- M2 (live progress): `cxx` CANNOT take a C++ `std::function` callback into Rust (C++→Rust fn pointers unimplemented). Reverse the wiring: an opaque C++ `ProgressSink` with methods `on_iteration(calls)/on_improvement` that Rust **calls out to** (Rust→C++ methods — supported) and C++ re-emits the coordinator’s `UpdateDisplay / UpdateOptimum=`. Must match C++ ordering exactly (30fps iteration, on-improvement) or the UI regresses.
- M3 (per-step from C++): expose `step()` that C++ drives in a loop; keeps progress control in C++ but diverges from the C++ Run structure. Not recommended for a drop-in.

7.2.1 Rust learning — callbacks, and why `cxx` flips the direction

M2’s `ProgressSink` may look odd if you come from C++/Python where a callback is just “a function you pass in”. Rust draws a sharp line between **callable you can hand out** and **callable you can accept**, which explains the whole M2 shape.

- **Book:** ch 13-01 (closures), ch 13 (Fn-traits), ch 16 (message passing/threads — the UI-pacing side).
- **rust-skills:** `closure-fn-trait-bounds` (`FnOnce` $\square$ `FnMut` $\square$ `Fn`; request the **least** restrictive), `closure-move-capture`, `api-builder-must-use`.

The ‘Fn’/‘FnMut’/‘FnOnce’ hierarchy is Rust’s way of saying **how many times and with what exclusivity** a closure may run:

- `FnOnce` — may be called once.
- `FnMut` — may be called repeatedly, may mutate captured state.
- `Fn` — may be called repeatedly, captures immutable (can be shared).

Normie-from-C++ (the host passes a `std::function` into Rust for progress):

```
1 // CANNOT work in cxx: C++->Rust fn pointers aren't implemented
2 opt.setProgressCallback([&int iter, double best]{ ... });
```

Why `cxx` forbids it: function pointers in `cxx` flow **Rust→C++** only. So the idiomatic M2 is the reverse — Rust calls a method on an opaque C++ object:

```
1 // Rust holds an opaque C++ ProgressSink and calls OUT (supported direction)
2 run_with_sink(&mut sink);           // sink.on_iteration(calls) etc.
```


The teaching moment: in C++/Python a callback is ambient (implicit, whatever the caller wants). In Rust a callback is a **first-class value/contract** with well-defined effects. When the boundary forbids one direction, you invert the ownership — the “sink” pattern (Rust owns a handle to the host and pokes it) is the idiomatic Rust way to express “the host wants progress while I run”.

M1’s simpler shape (run-to-completion, return `RunOutcome`) skips callbacks entirely — which is why it’s the recommended first milestone: it lets you get the algorithm + the cost crossing right before adding the progress-inversion complication. This sequencing — get the pure path correct, then layer the callback inversion — mirrors how Rust itself decouples pure logic from IO.

7.3 Cost invocation

- **Serial per-point:** keep scalar `evaluate6(x, 6doubles)` in a tight loop. Works, but crosses the cpp boundary per point (~20k–30k total evals).
- **Batch query** (matches C++ `BatchCostFunction`): collect new centers → one `&[Pose]` → `Vec<f64>` in input order. Recommended for real runs; maps onto cxx slices.
- **Slices:** cxx supports slices of shared structs (`Pose`) and primitives, NOT opaque types. `&[Pose]` ↔ C++ `=rust::Slice<const Pose>`. Prefer `&[Pose]` over a flat `Vec<f64>` for clarity.

7.3.1 Rust learning — slices, iterators, and “one FFI hop”

The batch-vs-serial decision is a place where idiomatic Rust and performance agree, and it teaches two habits: **slices over references-to-Vecs**, and **iterator pipelines over index loops**.

- **Book:** ch 04-03 (slices — ‘`&[T]`’ is a fat pointer: pointer + length), ch 13-02 (iterators), ch 13-04 (lazy iterators; collect only when needed), ch 08 (`Vec`).
- **rust-skills:** `own-slice-over-vec`, `perf-iter-over-index`, `perf-iter-lazy`, `perf-collect-once`, `anti-vec-for-slice`, `anti-index-over-iter`, `coll-map-choice`.

Normie-from-C++/Python (loop by index, pass `&Vec<f64>`):

```

1 fn batch_cost(costs: &Vec<f64>) -> Vec<f64> { // &Vec is a smell
2     let mut out = Vec::with_capacity(costs.len());
3     for i in 0..costs.len() { // indexing = bounds checks
4         out.push(costs[i] * 2.0);
5     }
6     out
7 }
```

Idiomatic (slice + iterator; accept ‘`&[f64]`’, chain lazily):

```

1 fn batch_cost(costs: &[f64]) -> Vec<f64> { // &[T], not &Vec<T>
2     costs.iter().map(|c| c * 2.0).collect() // lazy, no manual push
3 }
```

Why this matters for the BUFFER seam specifically:

- **‘`&[Pose]`’ vs empty-receiving ‘`Vec`’** — a slice is a *view* over contiguous memory; cxx maps ‘`&[T]`’ to C++ ‘`rust::Slice<const T>`’, which lets the C++ host pass a contiguous ‘`std::vector<Pose>`’ without copying. That’s the “zero-copy batched eval” you wanted.

- **One FFI hop beats many** — each scalar ‘evaluate6’ crossing is a boundary transit; a batch collects ~hundreds of new centers into one ‘&[Pose]’ and makes **one** call. For 20k-30k evals this is the difference between 20k crossings and a handful. The BatchCostFunction sibling in C++ exists for precisely this.
- **Iterator pipelines compose with your BTreeMap** — the POH gather you already wrote (‘self.bboxes.iter()...first_{keyvalue}()’) is a READ-ONLY borrow pipeline; the trisection (‘remove’/=pop_{first}=) is the MUTATING side. Slice-vector-boundary habits keep the borrows straight.

Reading tip: Brown Book ch 4-03’s slice diagram (the fat-pointer of ptr+len) is the exact mental model for ‘rust::Slice<const Pose>’.

7.4 Cost injection — Arg-time vs constructor-time

- A (cost via method arg): DirectOptimizer{boxes, current_best = } and =run(&mut self, cost: &CppCost) -> RunOutcome. Cost flows in at call time.
- B (cost stored in struct): constructor takes the handle; Rust owns it; run(&mut self) alone. Matches the C++ constructor shape.

Both work; A keeps the optimizer pure and the FFI thin.

7.4.1 Rust learning — the Cost capability as a trait (not a class)

Option A (cost via method arg) is the idiomatic one, and it’s worth understanding **why**: DirectOptimizer becomes a **pure data + algorithm** object and the cost is a small **capability** it knows how to call, injected per-call. In Rust that capability is best expressed as a **trait** — a contract, not a base class.

- **Book**: ch 10-02 (traits), ch 17-02 (trait objects dyn Trait), ch 13 (closures — the cost is eventually a closure over GPU state).
- **rust-skills**: trait-dyn-vs-generic, trait-object-safety, closure-fn-trait-bounds, closure-impl-fn-return, test-mock-traits.

The normie-from-C++ approach (a base class Cost with virtual eval(), or a std::function alias) reads as a concrete CppCost dependency everywhere:

```
1 // normie: hard-code the concrete handle into the optimizer
2 struct DirectOptimizer { cost: CppCost /* concrete */ }
```

Idiomatic (a trait that **both** a test stub and the real C++ handle implement):

```
1 trait Cost {
2     fn eval(&self, poses: &[Pose]) -> Vec<f64>; // batch shape, needed later
3 }
4 struct CppCostHandle(/* cxx opaque */); // impl Cost by calling evaluate6
5 struct AnalyticCost; // test stub for headless Tier-1
```

Making Cost a trait vs. a concrete CppCost dependency is what lets you run the SAME algorithm against a **test stub** and the **real GPU cost** — that’s test-mock-traits, and it’s exactly why the scaffold already sketches a trait Cost next to CppCost. Then the two shapes differ only by dispatch:

- **impl Fn (generics)** — zero-cost, monomorphized, great for a hot per-call closure. Use when the cost really is a closure.

- `dyn Fn=/trait object (=Box<dyn Cost>)` — one vtable, needed when the concrete cost is selected/stored at runtime. Your A/B already implies runtime selection.

The A/B switch is literally this: two backends are two implementations of one contract. Whether that contract is Rust's `Cost` trait (inside the crate) or C++'s `IOptimizer` virtual interface (across the boundary) is the same idea in two languages — which is what the `IOptimizer` option is pointing at.

7.5 Outcome/error channel — Result vs status

- `cxx Result` → C++ throws `rust::Error` (`std::exception`). In a Qt/CUDA pipeline, exceptions-on is uncertain → prefer **explicit status**: `RunOutcome { ok: bool (or code), best, best_cost, calls }`.
- Never let Rust panic across FFI (panic logs+aborts). Represent all error paths in the status struct / error enum; never unwrap across the boundary.

7.5.1 Rust learning — errors as values, not as side effects

The whole `Result` vs status-struct decision is a Rust-native philosophy: Rust has **no exceptions**. Errors are just values returned from functions. This changes the normie-from-C++/Python default dramatically.

- **Book**: ch 09-01 (panic), ch 09-02 (`Result` + ?), ch 09-03 (unwrap vs expect — when each is honest).
- **rust-skills**: `err-result-over-panic`, `err-thiserror-lib`, `err-anyhow-app`, `err-no-unwrap-prod`, `err-question-mark`, `err-context-chain`, `err-lowercase-msg`.

Normie-from-C++ (bool return + out-param, or throw an exception):

```
1 | bool Run(/* out */ double& best_cost);
```

Normie-from-Python (rely on raise and try/except):

```
1 | def run(...):
2 |     if bad: raise OptimizerError("bad")
```

Idiomatic Rust (return a `Result` / explicit status; the **caller** decides):

```
1 | type RunResult<T> = Result<T, OptimizerError>;
2 |
3 | #[derive(thiserror::Error, Debug)]
4 | pub enum OptimizerError {
5 |     #[error("all-zero range {range:?}")]
6 |     ZeroRange { range: Pose },
7 |     #[error("non-finite cost {cost}")]
8 |     NonFiniteCost { cost: f64 },
9 |     #[error("poisoned executor")]
10 |     Poisoned,
11 | }
```

Why the doc chose an **explicit status struct** (`RunOutcome{ ok: bool, ... }`) rather than Rust's `Result` for the FFI boundary specifically: `cxx` can map a Rust `Result` to a C++ 'rust::Error' exception — but that requires C++ exceptions to be **enabled**, which is uncertain in a Qt/CUDA toolchain, and a Rust panic crossing an FFI boundary is **undefined behavior** (it =abort()=s the whole process). So:

- **Inside** the Rust crate, use `Result<T, OptimizerError>` freely (=? everywhere) — that’s idiomatic and safe.
- **Across** the cxx boundary, return a plain status struct, and map the inner `Result` to it as the last step (‘out.ok = result.is_ok()’). This keeps the Rust-side ergonomics and the C++-side safety.

This two-tier pattern — rich errors inside, flattened explicit status at the FFI edge — is the idiomatic way to bridge Rust’s error values to a C++ caller:

the boundary is the ONE place you convert, and you never let a panic or a `rust::Error` exception escape across it.

7.6 Ownership / threading

- `Box<DirectOptimizer> → C++ rust::Box<DirectOptimizer>` (move-only, `unique_ptr`-like).
- `&self / &mut self` receivers allowed; `self` by value NOT. Multi-type extern-Rust must use explicit `self: &T`.
- `=CppCost` handle must out-live the run call (GPU captures).
- Serial evaluate from one worker thread.

7.6.1 Rust learning — ownership is the point (not a speed bump)

This whole section is Brown Book ch 4 made concrete. The normie-from-C++ reaction is “why do I need `Box<DirectOptimizer>`”, but in Rust `Box` is not a pointer-with-extras; it’s the **ownership** model made explicit.

- **Book:** ch 04-01 (what is ownership), ch 04-03 (slices), ch 15-01 (`Box` “for having a value on the heap”), ch 15 (smart pointers).
- **rust-skills:** own-borrow-over-clone, own-copy-small, own-move-large.

Normie-from-C++ (`unique_ptr` everywhere because it’s the default):

```
1 | fn make() -> Box<DirectOptimizer> { Box::new(DirectOptimizer::default()) }
```

...is actually fine **here** — because the C++ side owns the handle move-only, and `Box` is Rust’s `unique_ptr`-equivalent. The lesson: `Box` isn’t wrong because it’s a pointer; it’s wrong to reach for it when you only need to *borrow*. The doc’s `&self / &mut self` receivers are the borrow side.

Normie-from-Python mistake: passing `self` by value everywhere. cxx forbids ‘`self`’ by value for opaque types — and that’s not a limitation, it’s Rust refusing to let you silently *duplicate* something you can only own once. The distinction the section hammers (Brown ch 4-01):

- `T` (by value): I **move** it — I alone own it now.
- `&T`: I **borrow** it to read; others can too.
- `&mut T`: I **borrow** it to mutate; exclusive.

There is no “copy” in that list for a non-Copy type. That absence is the point. The rule “`CppCost` handle must out-live the run call” is likewise an ownership statement: C++ holds the `std::function+=GPU` captures; Rust borrows it for the duration of `=run`. The borrow ends when `run` returns, so the handle stays alive — the whole lifetime idea in one sentence.

7.7 Name / namespace

Use `#[cxx::bridge(namespace = "direct_rs")]` (or `cxx_name=/ =rust_name`) so `direct_rs::DirectOptimizer` `≠ ::jta::DirectOptimizer` and `direct_rs::Pose` stays distinct from the C++ `Point6D`.

8 Options for the coordinate A/B seam

8.1 Seam governance the C++ side offers

1. `OptimizerRunDriver` (factory-injected) — whole `OptimizerManager` lifecycle (stages + GPU + thread); for “alternative Manager”, not a single-stage optimizer-core drop-in.
2. `RunDirectStage` — single-stage optimizer kernel; the least-injection point for a Rust `DirectOptimizer` drop-in.
3. No existing switch: `direct_options` is a **variant** slot (algorithm knobs), not a backend selector, and fail-fast non-default fields.

8.2 Where backend state lives

- `OptimizerRunLaunch` / settings service: add `OptimizerBackend{ Cpp, Rust }`.
- NOT in `Options` (fail-fast); NOT in `OptimizerRunDriver` (over-scope).

9 Remaining open questions

- batch ownership: `rust::Slice<const Pose>` over a C++ contiguous vector (zero-copy) vs copying poses into Rust
- `--fno-exceptions` anywhere in Qt/CUDA toolchain? determines `Result` vs status-struct
- Rust `DirectOptimizer` unpin/sized; where the GPU context lives (keep single)
- first-fixtured scale (Tier-1 analytic golden) to gate Rust==C++ numeric parity
- callback ordering for M2 (iteration-vs-im-order per eval sequence)

10 Observations / conventions

- Keep the bridge minimal; real structs/methods stay native-side; the bridge is only value+pointer transit.
- Re-state contract details (eval ordering, budget) here; don’t rely on this handshake alone.
- Re-check file/line targets (`optimizer_manager.cpp:1303`, `cost.h`, `lib.rs`) before coding — they drift.
- Verify verbatim generated signatures (`rust::Box`, `::Slice`) with a tiny smoke crate first; don’t trust this doc’s shapes.

11 Review policy

This is a **reference handshake**, not an implementation plan. Decisions to double-check before coding:

1. `evaluate6` 6-double hot path vs shared-struct batch seam — which first?

2. A/B seam = RunDirectStage construction vs IOptimizer virtual adapter.
3. status-struct return vs `cxx Result` (exception).
4. M1-only vs live progress day-one for the first A/B test.

(Note: exact generated-header shapes for `rust::Box<T>` and `rust::Slice<const T>` were not verified verbatim from a build artifact; `cxx` latest ~1.0.194. Build a tiny smoke)

12 Appendix A — deep dive: borrowing and the DIRECT loop

If you only read one appendix, make it this one: it's ownership/borrowing made concrete against the exact shapes this project uses. The Brown Book ch 4 is the source; here we map it onto boxes: `BTreeMap<SizeKey, BTreeMap<CostKey, Hyperbox>>` and the run loop.

12.1 The four borrow flavors and when the loop uses each

Re-read this table whenever a borrow error confuses you. Every one of these is deliberately exercised by the DIRECT loop.

Rust expression	borrow kind	in this project
<code>&self / &T</code>	shared (read-only)	<code>gather_poh</code> over the whole tree
<code>&mut self / &mut T</code>	exclusive (write)	<code>trisect_and_reinsert</code> (it removes + inserts)
<code>T</code> by value	move (ownership)	<code>pop_first()</code> giving you the owned winner
<code>&[T]</code>	slice (a window)	<code>convex_hull(&[PohPoint])</code> — a borrowed view

The rule that governs almost every error here: **you can have many `&T` or one `&mut T`, never both for the same data at the same time.** (Brown ch 4-02.)

12.2 The borrow dance, zoomed to one iteration

Walk the loop and name the borrow at each breakpoint (this is Milestone 9's self-check, but written out):

```

1  // 1. GATHER — shared borrow: many readers, no writer
2  let mut candidate_points = self.gather_poh();      // &self
3  let selected = make_hull(&candidate_points);      // &[PohPoint]
4
5  // 2. TRISECT — here we switch from &self to &mut self
6  //   `self` was &self during gather; now we need &mut self to mutate the map.
7  for point in &selected {
8      // 3. TAKE — move ownership OUT of the map (needs &mut)
9      let winner: Hyperbox = {
10         let row = self.bboxes.get_mut(&OrderedFloat(point.size)); // &mut row
11         let (_key, hb) = row.expect("selected row exists").pop_first()
12             .expect("non-empty row");
13         hb // moves the Hyperbox out
14     }; // row borrow ends here
15
16     // 4. TRANSFORM — we OWN `winner` now; no borrow at all
17     let (center, [pos, neg]) = Hyperbox::trisect(winner);
18

```

```

19 // 5. REINSERT — move the children BACK in (&mut)
20 self.insert_box(center); // insert as known-cost
21 // pos/neg: batch-eval their centers, then insert with scores
22 }

```

The borrow transitions are the **spine** of the method:

- `&self->&mut self` happens by **method signature**, not by hand — a method

either takes `&self` or `&mut self`, so the compiler forces `gather` and `trisection` to be separate methods with different receivers. That’s why you can’t have one giant `&mut` method that gathers and mutates at once and expect the borrow rules to stay sane.

- The row borrow in step 3 is **scoped to the block**: the `&mut row` dies

at the closing brace, so `hb` (owned) can be returned out of it. This block- scoping is exactly the `mem-take-replace` / last-use pattern; the compiler ends the borrow at the winner’s last use.

12.3 The three errors you will actually hit (the canon)

These are the borrow-error codes you’ll encounter in this project, mapped to the *cause* and the *fix*:

Error	What it means	How it bites here	The fix
E0382	use of a moved value: you moved it away then used it again	reusing a Hyperbox after <code>pop_first()</code> moved it out	don’t reuse after move; or reorder so last use comes first
E0502	cannot borrow as mutable because it’s also borrowed as immutable (two borrows, wrong way)	<code>&mut self</code> while a <code>&self</code> borrow of the map is still alive	scope the shared borrow so it ends before the <code>&mut</code> begins
E0499	two mutable borrows at once	two live <code>&mut</code> into the tree (e.g. nested <code>get_mut</code> loops)	never hold two <code>&mut</code> into the same map; restructure
E0505	cannot move a value out of a borrow	<code>&self row</code> then trying to return an owned Hyperbox	<code>pop/remove</code> <i>out</i> via <code>&mut</code> or clone only the fields you need

- **Book:** the Brown Book has a whole chapter (ch 4) that animates these, with an interactive “editor” where you drag borrows around.
- **rust-skills:** `own-borrow-over-clone`, `own-move-large`, `mem-take-replace`, `anti-index-over-iter`.

Stop. **Check** — you hit “cannot move out of `*cost` which is behind a `&` reference” earlier. Which error code is that (E0505), and what’s the conceptual fix — borrow the `/fields/` you need, or move the whole value out via `=&mut`? It maps to “project the two numbers (Copy fields) rather than pulling the whole box.”

13 Appendix B — deep dive: ownership, lifetimes, and Send/Sync

This appendix is for the moments when `value/move/borrow` isn’t enough — when you need `Box`, when a lifetime shows up in a signature, and when the compiler starts asking about `Send` and `Sync` because of the worker thread + GPU cost.

13.1 Lifetimes: what they really are (and aren't)

A lifetime is not a value you pass around; it's the compiler's label for "how long a borrow is valid". You mostly write `&T` and let Rust elide lifetimes; an explicit lifetime (`<'a>`) appears when a return type borrows from an argument.

Normie-from-C++ confusion: "`&Point6D` is a const reference, why do I need a lifetime?" The answer: the returned value's lifetime is tied to the borrow's origin. In this project you rarely write explicit lifetimes because you *own* the values. The one place they appear is `cxx` opaque types and `&CppCost` borrows — which `cxx` manages for you.

- **Book:** ch 10-03 (lifetimes), Brown ch 4-02 (the borrow table).
- **rust-skills:** `own-lifetime-elision`, `type-phantom-marker`.

13.2 Box: only when you need a stable address or heap

Recall `rust::Box<DirectOptimizer>` on the C++ side. `Box` is one specific tool, not a default. You reach for `Box<T>` when:

- `T` is recursive (a tree node) — not here.
- `T` is huge and you want to move a pointer instead — your `Hyperbox` is tiny.
- you need heap because of a trait object `Box<dyn Cost>` — only if you pick runtime dispatch for the cost (Milestone 6).
- FFI wants a stable pointer (`cxx Box<DirectOptimizer>`) — THIS is your case.

Normie mistake: `Box` every struct "just in case". Stop — value `Hyperbox`='s by default; `box` only the FFI boundary object. `=own-move-large, mem-box-large-` variant.

13.3 Send and Sync: why the GPU cost blocks one thread

This is where `Send` and `Sync` finally matter. The worker thread (on the C++ side) calls `run` synchronously; the GPU cost (`:CppCost`) mutates `GPUModel` pose state on each evaluate. So:

- `Send` means "value can be transferred to another thread." `CppCost` wraps a `std::function` + GPU captures — Rust can't prove it's thread-safe, so it won't be `auto-Send`.
- `Sync` means "&T can be shared across threads." Same story.

Rather than fight that, the design keeps everything on ONE worker thread and calls `run` to completion (M1). That sidesteps `Send` / `Sync` entirely — you never move `CppCost` between threads because the C++ side owns the thread and the handle. This is why the doc's threading rule is "serial, one thread" and why `=M1` (run-to-completion) is recommended before `M2` (which needs care about which thread calls into the sink).

- **Book:** ch 16 (concurrency — `Send`/`Sync` are the runtime of fear free threading; `Rc` vs `Arc`).
- **rust-skills:** `own-arc-shared` (when you DO need shared ownership across threads, upgrade `Rc`=>`Arc`), `conc-thread-local`, `unsafe-send-sync-manual`.

13.4 Rc/=Arc/=RefCell: the “when you DON’T need them” warning

If you find yourself reaching for `Rc`, `Arc`, or `RefCell` in `DIRECT`, pause hard. They are for *shared* or *interior* ownership, and the whole design avoids both:

- `Rc/Arc` for shared ownership — you never share a `Hyperbox`; the map owns it and you move it in/out. You don’t need shared pointers at all.
- `RefCell` for interior mutability — you mutate through `&mut` methods, not through interior mutation. If you’re grabbing `RefCell`, you’ve probably mis-structured a method’s receiver (should be `&mut self`, not `&self + a hidden RefCell`).

Intro Section n/a; see Interior Mutability appendix.

Stop. **Check** — why is `CppCost` not `auto-Send/Sync`, and how does keeping the run on one thread dodge that (instead of adding `unsafe impl Send`)? Because Rust can’t verify the captured GPU state is thread-safe; single-threading makes the question moot without any `unsafe`.

14 Appendix C — interior mutability: when you do NOT need it

Interior mutability (`Cell`, `RefCell`, `Mutex`, `RwLock`, `Atomics`) lets you mutate through a shared reference. It is a **workaround for a borrow it gives up**, and `DIRECT`’s design exists largely to not need it. This appendix helps you recognize the moment you’re reaching for it and whether that’s a real need or a sign you’ve mis-structured.

14.1 The honest use case

You need interior mutability when: a value must be mutably changed, but you only have `&self != &` access, and you can’t re-structure to `&mut`. Classic examples: caches, counters behind an API that only hands out `&self`, or raw pointer mutation like GPU buffers owned by C++ and visible to Rust as `&T`.

In `DIRECT`, the places you might ‘tempt’ reach for it:

- a `cost_function_calls_counter` you bump in `&self` methods;
- a cache of computed `size()` values;
- the `CppCost` handle that mutates GPU state but comes in as `&CppCost`.

14.2 Why `DIRECT` mostly dodges it

1. **The counter.** In the C++ it’s a member bumped by all methods. The Rust idiomatic move is to make `&mut`-taking methods the **ONLY** mutators, and keep the counter next to boxes behind `&mut self`. If a counter needs to be readable from `&self` AND written from elsewhere, `Cell<u32>` is legitimate — but only because you’ve proven `&self` must observe it while `&mut` writes elsewhere. Often you can reorder so a single `&mut self` method owns the bump and the read, and `Cell` never appears.
2. **The cache.** `perf-profile-first`: don’t cache until the profiler says so. And even then, a `RefCell` cache turns every read into a potential panic (`.borrow()` twice = panic) — you trade one class of bug for another.

3. **The GPU CppCost.** This is the real one. On the C++ side, `evaluate()` mutates `GPUModel` pose via a mutable closure, but to Rust it arrives as `&CppCost`. If Rust needed to *store* that handle and call it from multiple places, you might reach for `RefCell<&CppCost>`. But the design passes `= &CppCost =` /in to `run=/batch=` and finishes the call synchronously, so the C++ side alone owns mutation — no interior mutability needed in Rust.

14.3 The “is it a real need?” checklist **

Ask, in order, before introducing `RefCell` / `Cell`:

1. Can I change a `&self` method to `&mut self`? If a mutator touches state, it should *take* `&mut` — that’s the honest signal.
2. Can I return data instead of mutating in place? (gather returns `Vec<PohPoint>`; it doesn’t scribble a member.)
3. Is this genuinely shared mutable state visible across `&` reads and `&mut` writes? Only *then* is `Cell` `!=` `RefCell` `!=` `Mutex` justified, and pick `Mutex` if it crosses threads.

If the answer to #1 or #2 is yes, you don’t need interior mutability — you need a better method signature. Reaching for `RefCell` early is the “masturbatory types” spike this project wants to avoid (see `anti-over-abstraction` and the `Options-as-data` block).

- **Book:** ch 15-05 (`RefCell` “interior mutability”), ch 16 (threads = `Mutex` vs `RwLock`), `rust-skills own-refcell-interior =`

`own-mutex-interior.`

- **rust-skills:** `own-refcell-interior`, `own-mutex-interior`, `own-rc-single-thread`, `anti-over-abstraction`.

Stop. **Check** — you’re about to add `RefCell<u32>` for a call counter. Run the checklist: can the bumping method take `&mut self`? If yes, drop the `RefCell`. Only keep it when `&self` readers **MUST** see the current count while another object owns mutation — which, in `DIRECT`, the `&mut self`-run loop handles without interior mutability.

15 Appendix D — Option and Result in practice (the Python jump)

Coming from Python, this is the appendix that will save you the most time. Python has `None` and exceptions and they’re **ambient**: any value might be `None`, any call might raise. Rust makes both into ordinary types you must open before use. `Option<f64>` is not “an `f64` that might be missing” — it is a different type from `f64`, and the compiler will not let you add 1.0 to it.

This is exactly the `cost_at_center`: `Option<f64>` field from Step 3, so every example below is real.

- **Book:** ch 06-02 (`match` + `Option`), ch 06-03 (`if let`), ch 09-02 (`Result` + `?`), ch 18 (patterns in depth).
- **rust-skills:** `type-option-nullable`, `err-question-mark`, `err-no-unwrap-prod`, `pat-exhaustive-enum`.

15.1 The four ways to open an Option (in order of preference)

```

1 // 1. match – when both arms do real work
2 match hb.cost_at_center {
3     Some(c) => record(c),
4     None    => queue_for_eval(hb),
5 }
6
7 // 2. if let – when only one arm does work
8 if let Some(c) = hb.cost_at_center { record(c); }
9
10 // 3. let-else – when None means "bail out of this function" (the flattener)
11 let Some(c) = hb.cost_at_center else { return Err(OptimizerError::Unscored) };
12 // `c` is a plain f64 from here down – no nesting, no indentation creep
13
14 // 4. combinators – when you're transforming, not branching
15 let doubled: Option<f64> = hb.cost_at_center.map(|c| c * 2.0);
16 let shown: f64          = hb.cost_at_center.unwrap_or(f64::INFINITY);

```

let-else (option 3) is the one C++/Python developers never discover on their own and then use constantly. It is the idiomatic early-return: the happy path stays un-indented, the failure path is one line. Reach for it whenever your match has a None arm that just leaves.

15.2 The combinator cheat-sheet

These are the Option/=Result methods worth memorizing. Every one of them replaces an if you'd have written in C++.

Call	Means	Use here
.map(f)	transform the inside, keep the wrapper	scale a cost, keep “unscored”-ness
.and_then(f)	map where f itself returns Option/Result	chained lookups (row → box)
.unwrap_or(v) / _else(f)	supply a default	treat unscored as INFINITY for sorts
.ok_or(e) / _else(f)	Option → Result (attach a reason)	“selected row missing” → error
.ok() / .err()	Result → Option (throw the reason away)	the FFI-edge flatten
.filter(p)	keep Some only if predicate holds	“scored AND finite”
.take()	move the value out, leave None behind	claim a pending cost exactly once
.as_ref() / .as_mut()	&Option<T> → Option<&T>	peek without moving out of &self
.is_some_and(p)	test the inside without unwrapping	guards in if
.zip(other)	pair two Some into one Some (a, b)	need both center costs or neither

.take() deserves a flag: it is mem-take-replace in miniature and it's how you move a non-Copy value out of a struct field you only have &mut to. When the borrow checker says “cannot move out of self.field”, .take() is usually the answer.

15.3 ? — the operator that makes Result bearable

Normie (Python habit: let it raise, or C++ habit: check every return):

```

1 let row = match self.bboxes.get_mut(&key) {
2     Some(r) => r,
3     None => return Err(OptimizerError::MissingRow { key }),
4 };
5 let (_k, hb) = match row.pop_first() {
6     Some(p) => p,
7     None => return Err(OptimizerError::EmptyRow { key }),
8 };

```

Idiomatic (? = “unwrap or return the error to my caller”):

```

1 let row = self.bboxes.get_mut(&key).ok_or(OptimizerError::MissingRow { key })?;
2 let (_k, hb) = row.pop_first().ok_or(OptimizerError::EmptyRow { key })?;

```

Six lines to one, with the **same** control flow and no hidden unwinding. This is what “errors are values” buys you: ? is sugar for an early return, nothing more. It works on Option too (returns None), which is why small helper functions often return Option<T> just to get ?.

15.4 unwrap vs expect vs neither

- .unwrap() — never in this crate’s non-test code. It’s a panic, and Appendix B tells you a panic across FFI aborts the process.
- .expect("6-element array") — acceptable **only** when the invariant is structurally guaranteed, and the string says **why**. The min_by_key call in Step 7 is the legitimate case: a [u32; 6] is never empty.
- Anything else — ? or an explicit match.

Stop. **Check** — you have Option<f64> and need an f64 to compare. Name three different correct moves and when each is right. (unwrap_or(INFINITY) if unscored should sort last; let-else if unscored is a bug worth returning on; is_some_and(|c| c < best) if you’re only testing.)

16 Appendix E — the run loop assembled (contract-faithful)

Milestone 9 says “wire it together” and the doc never shows the wiring. Here it is, and the reason it’s worth spelling out is that **the eval-ordering contract from “Contract being filled” lives in exactly one function** — get it wrong and the Tier-1 golden diverges in a way that’s miserable to bisect.

Re-read the contract line first:

> eval ordering: calls++ → non-finite → optimum update → storage → improvement callback

16.1 The struct

```

1 pub struct DirectOptimizer {
2     tree: BoxTree,           // the two-level BTreeMap (Step 4)
3     budget: u32,             // cumulative cap: trunk 20k → branch 25k/30k → leaf 35k
4     call_offset: u32,        // SetCallOffset – cumulative across stages
5     calls: u32,
6     non_finite: u32,
7     best: Pose,
8     best_cost: f64,
9     stop: Arc<AtomicBool>,   // cooperative Stop(), set from the C++ thread
10 }

```

That `Arc<AtomicBool>` is the **one** honest interior-mutability case in the whole design, and it’s worth pausing on — Appendix C told you to be suspicious of `RefCell`, and this is the exception that proves the rule. `Stop()` is called from a different thread than `run()`, so there is no way to restructure it into `&mut self`: the checklist’s question 3 (“genuinely shared mutable state across threads?”) answers **yes**. `AtomicBool` not `Mutex<bool>` because it’s one bit and never needs to be read-and-written atomically together with anything else. Everything else in the struct stays behind `&mut self`.

16.2 The eval helper — where the contract is enforced

```

1 /// Evaluate a batch and fold the results into optimizer state.
2 /// Returns costs in INPUT ORDER (the contract the C++ BatchCostFunction keeps).
3 fn eval_batch(&mut self, cost: &dyn Cost, poses: &[Pose]) -> Vec<f64> {
4     let costs = cost.eval(poses);           // ONE FFI hop for the whole batch
5     debug_assert_eq!(costs.len(), poses.len(), "batch cost must preserve length");
6
7     for (pose, &c) in poses.iter().zip(costs.iter()) {
8         self.calls += 1;                     // 1. calls++ (counts non-finite too)
9         if !c.is_finite() {
10             self.non_finite += 1;           // 2. non-finite
11             continue;                       // a non-finite eval never becomes best
12         }
13         if c < self.best_cost {              // 3. optimum update
14             self.best = *pose;
15             self.best_cost = c;
16             self.improved_this_iter = true; // 5. deferred – see below
17         }
18     }
19     costs                                   // 4. storage is the caller's job (reinsert)
20 }

```

The subtle bit: the contract puts **storage before the improvement callback**, but the improvement is **detected** here during the eval. So you don’t fire the sink here — you set a flag and fire it after reinsertion. Firing early is the classic port bug: the UI shows a new optimum for a box that isn’t in the tree yet, and if anything downstream reads the tree on that signal, it races.

16.3 The loop

```

1 pub fn run(&mut self, cost: &dyn Cost, sink: Option<&mut dyn ProgressSink>) -> RunResult<()> {

```

```

2   if self.tree.is_empty() { return Err(OptimizerError::ZeroRange { range: self.range }); }
3
4   while (self.calls + self.call_offset) < self.budget
5       && !self.stop.load(Ordering::Relaxed)
6   {
7       self.improved_this_iter = false;
8
9       // GATHER — &self borrow, no mutation (Step 5)
10      let points = self.tree.gather_poh();
11      let selected = convex_hull(&points);           // pure (Step 6)
12      if selected.is_empty() { break; }             // nothing left to split
13
14      // TRISECT — &mut self; take out, transform, put back (Step 7)
15      let mut pending: Vec<Pose, PendingSlot> = Vec::new();
16      for point in &selected {
17          let winner = self.tree.take_min_of_row(point.size)?; // owned
18          let (center, [pos, neg]) = Hyperbox::trisect(winner);
19          self.tree.insert_scored(center);                // cost inherited
20          pending.push((pos.center, PendingSlot::Pos(pos)));
21          pending.push((neg.center, PendingSlot::Neg(neg)));
22      }
23
24      // ONE FFI hop for every new center this iteration
25      let poses: Vec<Pose> = pending.iter().map(|(p, _)| *p).collect();
26      let costs = self.eval_batch(cost, &poses);
27
28      // STORAGE — then and only then, the callbacks
29      for ((_, slot), c) in pending.into_iter().zip(costs) {
30          self.tree.insert_with_cost(slot.into_box(), c);
31      }
32      if let Some(s) = sink.as_deref_mut() {
33          s.on_iteration(self.calls + self.call_offset); // ~30fps pacing is C++'s job
34          if self.improved_this_iter { s.on_improvement(self.best, self.best_cost); }
35      }
36  }
37  Ok(())
38 }

```

And the FFI edge — the one place Result becomes status (M8):

```

1  pub fn run_ffi(&mut self, cost: &CppCost) -> ffi::RunOutcome {
2      let handle = CppCostHandle(cost);
3      let result = self.run(&handle, None);
4      ffi::RunOutcome {
5          best:      self.best,
6          best_cost: self.best_cost,
7          calls:     self.calls,
8          ok:        result.is_ok(), // rich error → one bool, deliberately
9      }
10 }

```

16.4 Three things this skeleton exposes that the prose didn't

1. **Batch granularity is per-iteration, not per-box.** All the new centers from one POH round go in one `&[Pose]`. That's the "one FFI hop" claim made concrete: $\sim 2 \times |\text{hull}|$ poses per crossing instead of one.
2. **The budget can overshoot.** `eval_batch` evaluates the whole batch, so the final iteration may push calls past budget. The C++ checks the guard **between** iterations too — so this matches — but confirm it against `direct_optimizer.h`, because if C++ truncates mid-batch and Rust doesn't, the golden diverges on call count. **This belongs in "Remaining open questions".**
3. **sink is `Option<&mut dyn ProgressSink>`.** M1 passes `None` and the whole progress path costs nothing; M2 passes `Some`. One signature, both milestones — this is `trait-dyn-vs-generic` paying off.

Stop. **Check** — trace the calls counter for one iteration with a 4-box hull. How many FFI crossings, how many calls increments, and in what order does a new best reach the UI?

17 Appendix F — testing the port: unit, property, parity

A port has one job: **be the same**. That makes this the appendix with the highest ratio of value to Rust-novelty, because Rust's test story is unusually good and you can get parity confidence before any C++ is linked.

- **Book:** ch 11 (all of it — it's short), ch 11-03 (integration tests).
- **rust-skills:** `test`, `mock`, `traits`, `err`, `result`, `over-panic`, `perf`, `profile`, `first`.

17.1 Tier 0 — unit tests live next to the code

```

1 // bottom of src/poh.rs
2 #[cfg(test)]                                // compiled ONLY under `cargo test`
3 mod tests {
4     use super::*;
5
6     #[test]
7     fn hull_of_a_known_set() {
8         let pts = vec![p(1.0, 5.0), p(2.0, 3.0), p(3.0, 4.0), p(4.0, 1.0)];
9         let hull = convex_hull(&pts);
10        assert_eq!(hull.len(), 3);
11        assert!(hull.windows(2).all(|w| w[0].size < w[1].size)); // monotone in size
12    }
13
14    #[test]
15    fn trisect_preserves_depth_budget() {
16        let parent = Hyperbox { cost_at_center: Some(1.0), center: Pose::ZERO, depths: [0; 6]
17        ↪ };
18        let (center, [pos, neg]) = Hyperbox::trisect(parent);
19        assert_eq!(center.depths.iter().sum:::<u32>(), 1);
20        assert!(pos.size() < 1.0 && neg.size() < 1.0);
21        assert!(pos.cost_at_center.is_none()); // not scored yet - Step 3's Option
22    }
23 }
```

The `#[cfg(test)] mod tests` idiom is worth internalizing: tests sit in the **same file** as the code and can see private items. There is no separate test tree to keep in sync, and no header to expose internals through. This is one of the places where Rust is simply nicer than C++ and you should exploit it — every Step in this doc should end with a test module.

17.2 Tier 1 — the analytic golden, headless

This is why `Cost` is a trait (M6). With a stub cost you can run the **entire** algorithm with no GPU, no Qt, no C++:

```

1  struct Sphere;                                // f(x) = sum(x_i^2), min at origin
2  impl Cost for Sphere {
3      fn eval(&self, poses: &[Pose]) -> Vec<f64> {
4          poses.iter()
5              .map(|p| p.x*p.x + p.y*p.y + p.z*p.z + p.xa*p.xa + p.ya*p.ya + p.za*p.za)
6              .collect()
7      }
8  }
9
10 #[test]
11 fn finds_the_origin_within_budget() {
12     let mut opt = DirectOptimizer::new(range(), start(), 20_000, PohMode::ConvexHull);
13     opt.run(&Sphere, None).unwrap();
14     assert_relative_eq!(opt.best_cost, 0.0, epsilon = 1e-6);
15     assert!(opt.calls <= 20_000);
16 }

```

17.3 Tier 2 — property tests for the invariants prose can't hold

`proptest` generates hundreds of inputs and shrinks failures to a minimal case. Use it for the claims this doc makes in English:

```

1  proptest! {
2      // the hull is always a subset of its input
3      #[test]
4      fn hull_is_subset(pts in prop::collection::vec(any_poh_point(), 1..50)) {
5          for h in convex_hull(&pts) { prop_assert!(pts.contains(&h)); }
6      }
7
8      // trisection never loses volume: children's extents tile the parent's
9      #[test]
10     fn trisect_conserve(depths in any::<[u32; 6]>()) {
11         let parent = Hyperbox { cost_at_center: Some(0.0), center: Pose::ZERO, depths };
12         let (c, [p, n]) = Hyperbox::trisect(parent);
13         prop_assert_eq!(c.depths.iter().sum::<u32>(),
14             depths.iter().sum::<u32>() + 1);
15         prop_assert!(p.size() <= parent_size && n.size() <= parent_size);
16     }
17 }

```

Property testing is not a Rust-specific idea, but Rust's `proptest` is where most people meet it, and “the same optimum for the same cost sequence” is precisely a property, not an example.

17.4 Tier 3 — parity against the C++ (the actual gate)

The contract says the judge is “the SAME optimum for the same cost sequence”. So make the cost sequence the artifact:

1. Instrument the C++ `DirectOptimizer` to dump, per eval: index, the 6 doubles in, the cost out. Run one fixture. That file is your golden.
2. In Rust, implement `Cost` as a **replay** over that file: it asserts the incoming pose matches the recorded pose and returns the recorded cost.
3. Any divergence fails at the **first differing eval**, with an index — instead of you comparing two final optima and having no idea where they split.

```
1 struct ReplayCost { records: Vec<(Pose, f64)>, cursor: Cell<usize> }
2 // eval(): for each pose, assert_relative_eq! against records[cursor], return cost
```

This is the single highest-leverage test in the port, and note what it needs: `Cost` being a trait, and the batch/serial distinction being a detail the trait hides. Two design decisions from M6 exist mostly to make this test possible.

(`Cell<usize>` for the cursor is legitimate here — Appendix C’s checklist, question 3, with `&self` forced on you by the trait signature. A test is also the one place a `RefCell` panic is acceptable: it fails the test, which is the point.)

Stop. **Check** — which tier catches “Rust picks a different tie-break when two boxes have equal cost”? (Tier 3, immediately, at the first tie — and Tier 2 if you write the property. Tier 1 might never notice, because the final optimum can match by luck.)

18 Appendix G — C++ → Rust and Python → Rust translation tables

A lookup for the “what’s the Rust word for this” moment. Use it to stop searching, not to stop thinking — the third column is where the difference lives.

18.1 From C++

C++	Rust	The difference that matters
<code>std::vector<T></code>	<code>Vec<T></code>	<code>Vec</code> is move-only unless <code>T: Clone</code> ; no copy ctor
<code>const std::vector<T>& arg</code>	<code>&[T]</code>	slice works for arrays/ <code>Vec</code> /boxed alike
<code>std::array<T, N></code>	<code>[T; N]</code>	size is part of the type, same as C++
<code>std::unique_ptr<T></code>	<code>Box<T></code>	no null state; <code>Option<Box<T>></code> if you need one
<code>std::shared_ptr<T></code>	<code>Rc<T> / Arc<T></code>	<code>Rc</code> is single-thread only; compiler enforces it
<code>T* (nullable, non-owning)</code>	<code>Option<&T></code>	no null deref possible
<code>std::optional<T></code>	<code>Option<T></code>	<code>Option</code> must be opened; no implicit bool
<code>std::variant<A, B></code>	enum with data	exhaustive match, no <code>std::visit</code> ceremony
<code>std::map<K, V></code>	<code>BTreeMap<K, V></code>	needs <code>K: Ord</code> — hence the whole Step 4
<code>std::unordered_map<K, V></code>	<code>HashMap<K, V></code>	needs <code>K: Hash + Eq</code> ; not sorted
<code>std::function<R(A)></code>	<code>Box<dyn Fn(A) -> R></code>	closures are types; <code>Fn</code> tier is explicit
abstract base + virtual	<code>trait + dyn Trait</code>	no inheritance of data , only of contract
<code>template <typename T></code>	<code>generic <T: Bound></code>	bounds checked at definition, not instantiation
<code>const& member fn</code>	<code>&self</code>	borrow checker enforces it, not convention
mutable member	<code>Cell / RefCell</code>	opt-in and visible in the type (Appendix C)
<code>throw / try</code>	<code>Result + ?</code>	no unwinding across FFI; errors are return values
<code>std::move(x)</code>	<code>just x</code>	move is the default; no moved-from husk to read
RAII destructor	<code>Drop</code>	same idea; runs at last use, not scope end (NLL)
<code>assert</code>	<code>assert! / debug_assert!</code>	<code>debug_assert!</code> compiles out in release

18.2 From Python

Python	Rust	The difference that matters
<code>list</code>	<code>Vec<T></code>	one element type, no mixing
<code>dict</code>	<code>HashMap / BTreeMap</code>	key needs <code>Hash+Eq</code> or <code>Ord</code>
<code>tuple</code>	<code>(A, B)</code>	fixed arity in the type
<code>None</code>	<code>Option::None</code>	a different type , not a value of every type
<code>raise / except</code>	<code>Result + ?</code>	callers see failure in the signature
duck typing	<code>trait bounds</code>	the contract is written down and checked
<code>@dataclass</code>	<code>struct + #[derive(...)]</code>	derives are opt-in per capability
list comprehension	<code>.iter().map().filter().collect()</code>	lazy until <code>collect</code>
<code>for i in range(len(x))</code>	<code>for item in &x</code>	idiomatic Rust rarely indexes (<code>perf-iter</code>)
<code>copy.deepcopy</code>	<code>.clone()</code>	explicit, visible, and usually avoidable
GIL / “just use threads”	<code>Send / Sync</code>	data races are compile errors (Appendix B)
<code>isinstance</code> dispatch	<code>match on an enum</code>	exhaustive; new variant = compile error at e

The Python row worth staring at is the last one. In Python, adding a case means hunting for every `isinstance` chain. In Rust, adding an enum variant makes every non-wildcard match fail to compile until you handle it. That is the single biggest reason this doc keeps pushing enum over `u8` — it’s not tidiness, it’s a refactoring superpower.

19 Appendix H — debugging the FFI seam

Everything above assumes things work. This is the appendix for 3pm on the day the linker starts shouting. Rust’s compile errors are excellent; cxx’s and the linker’s are not, so pattern-matching on the symptom is the fastest route.

19.1 First: look at what cxx generated

Two commands answer most questions:

```
1 | cargo expand --lib ffi           # the Rust side the bridge macro wrote
2 | cat target/*/cxxbridge/direct-rs/src/lib.rs.h # the C++ side it wrote
```

Every “is it `rust::Box<T>` or `rust::box<T>`”, does the slice come in as `rust::Slice<const Pose>`” question in this doc is answered by that header, for **your** cxx version, in seconds. This is the smoke-crate advice from “Review policy”, but you don’t need a smoke crate — you need to read the artifact.

19.2 Symptom → cause table

Symptom	Cause
undefined reference to <code>direct_rs::new_direct_optimizer</code>	staticlib not linked, or name mangling differs
undefined reference to <code>CppCost::evaluate6</code>	declared in bridge, not defined in C++
cxx: “unsupported type” on a bridge signature	tried to pass a non-shared, non-opaque type
cxx: “self must be a reference”	<code>fn run(self, ...)</code> on an opaque type
cxx: function pointer / callback rejected	C++→Rust fn pointers unimplemented
process dies with thread panicked ... note: aborting	a panic reached an extern boundary
C++ sees garbage in a Pose field	field order differs between the two decls
optimum matches, calls doesn’t	budget truncation differs mid-batch
Pose values fine, costs all inf	handle outlived / GPU state not set up
works in debug, wrong in release	UB, or an f64 comparison <code>/=debug_assert!=</code>

19.3 Tools worth having configured before you need them

- `RUST_BACKTRACE=1` — set it in the launch environment permanently. A Rust panic without a backtrace tells you almost nothing.
- `nm -C libdirect_rs.a | grep direct_rs` — proves the symbol exists before you blame CMake.
- `gdb / lldb` — both step across the boundary; Rust frames show up with mangled-but-readable names. Breaking on `rust_panic` catches the abort case **at the panic**, not after.
- `RUSTFLAGS“-Z sanitizer=address”` (nightly) — worth one run once the batch slice path is live, because that’s the only place raw memory is shared.
- `cargo tree` — when two crates disagree on a cxx version, which produces spectacularly confusing layout mismatches.

19.4 The panic firewall (do this once, in `lib.rs`)

Because a panic crossing FFI is undefined behavior and the design says “never let Rust panic across FFI”, make that structural rather than aspirational:

```

1 fn run(&mut self, cost: &CppCost) -> ffi::RunOutcome {
2     std::panic::catch_unwind(std::panic::AssertUnwindSafe(|| {
3         self.run_inner(cost)
4     })))
5     .unwrap_or_else(|_| ffi::RunOutcome { ok: false, ..Default::default() })
6 }
```

Caveats worth understanding rather than copying blindly: `catch_unwind` does nothing under `panic=abort`, and `AssertUnwindSafe` is you promising the state isn’t left half-mutated. It’s a net, not a design. The design is `Result` everywhere inside (Appendix D) — this catches the bug you didn’t predict, on a customer’s machine, instead of aborting the whole application.

- **Book:** ch 09-01 (panic and unwinding), ch 19-01 (=unsafe=/FFI).
- **rust-skills:** `err-no-unwrap-prod`, `unsafe-safety-comment`.

Stop. **Check** — the C++ side reports a plausible optimum but `out.ok = false` every run. Where do you look first? (The `Result=>ok` flatten in `run_ffi`: something inside is returning `Err` **after** best was already updated — which is exactly the information the flatten throws away. Log the `OptimizerError` on the Rust side before you drop it; this is the “what is lost at the edge” from M8, biting.)

20 Rust idiom glossary + Brown Book index

A cross-reference table: the concept this FFI handshake touches, where to read it in the Brown Book, the rust-skills rule, and a one-line “normie → idiomatic” teaser. Use it to jump straight to the depth you need.

Rust concept	Brown Book	rust-skills rule	normie → idiomatic guardrail
ownership / move semantics	ch 04-01	own-borrow-over-clone	“don’t copy, don’t pass by value — move or borrow
borrowing = <code>&T</code> / <code>&mut T</code>	ch 04-02	own-borrow-over-clone	“ <code>&mut</code> is exclusive; <code>&</code> is shared”
slices = <code>&[T]</code> (fat pointer)	ch 04-03	own-slice-over-vec	“accept <code>&[T]</code> , not <code>&Vec<T></code> ”
structs (value types)	ch 05	own-copy-small	“small POD-alleys by value; big/owned on heap”
enums + exhaustive match	ch 06	type-enum-states	“use enum for mutually-exclusive states, not <code>u8</code> ”
<code>Result</code> / <code>?</code> / no exceptions	ch 09	err-result-over-panic	“errors are values; caller decides”
<code>Box<T> ≈ unique_ptr</code>	ch 15-01	own-move-large	“only box when you need heap or a stable address”
traits as contracts	ch 10-02	trait-dyn-vs-generic	“ <code>Cost</code> is a trait, not a base class”
<code>dyn Trait</code> / trait objects	ch 17	trait-object-safety	“use <code>dyn</code> only when runtime dispatch is needed”
closures: <code>Fn</code> / <code>FnMut</code> / <code>FnOnce</code>	ch 13	closure-fn-trait-bounds	“request the least-restrictive <code>Fn</code> you need”
iterators (lazy, chainable)	ch 13	perf-iter-over-index	“ <code>map/filter/collect</code> , not index loops”
unsafe + FFI / <code>#[repr(C)]</code>	ch 19	unsafe-safety-comment	“boundary-only; never leak a panic”
error type with <code>thiserror</code>	ch 09-02	err-thiserror-lib	“derive <code>Error</code> ; use <code>?</code> everywhere inside”
panic = abort across FFI	ch 09-01	err-no-unwrap-prod	“unwrap only on verified invariants”
newtype for IDs/units	ch 19-03?	type-newtype-ids	“ <code>BoxBudget(u32)</code> , not bare <code>u32</code> ”
Default derive for options	ch 05	api-default-impl	“ <code>Default == C++ default ctor</code> ”

Key Brown Book sections to open (each is a chapter page; substitute the number into <https://rust-book.cs.brown.edu/chNN-0X-...html>):

1. `ch04-02-references-and-borrowing.html` — the borrow table (move/&T/&mut T).
2. `ch09-02-recoverable-errors-with-result.html` — `Result` + ?.
3. `ch10-02-traits-defining-shared-behavior.html` — the `Cost` trait.
4. `ch13-01-closures.html` — `Fn` traits + capture.
5. `ch15-01-box-point-to-data-on-the-heap.html` — why `Box` here.
6. `ch19-01-unsafe-rust.html` — what `unsafe` extern gives up.

How to grow with this doc: whenever you implement a piece and hit a borrow checker error, look it up in the glossary’s “normie → idiomatic” column first; the Brown chapter it points to walks through the visual/physical model, and the rust-skills rule gives the sharper 1-3 sentence rule. Re-read ch 4 after every few-up milestone — ownership is the book’s spine, and the FFI handshake above is one long exercise in it.

20.1 Borrow-checker error lookup (keep this beside the Brown ch 4 quizzes)

The fastest way to grow from each compiler rejection is to map the error to a cause and a move. This is the same canon as Appendix A, but organized as a diagnostic cheat-sheet with the *normie* reflex you’ll have and the *idiomatic* response.

Error code	You (normie) wrote	What Rust means	The idiomatic response
E0382	reuse a <code>Hyperbox</code> after <code>pop_first()</code> moved it	“use of moved value” — you gave it to the map / to trisect	reorder so you still need it
E0502	<code>&self</code> borrow alive while you call <code>=&mut self</code> =-mutating method	“cannot borrow as mutable; it’s also borrowed as immutable”	scope the mutable borrow into a <code>&=scope</code>
E0499	two live <code>&mut</code> into the tree (nested <code>get_mut</code> loop)	“cannot borrow as mutable more than once”	restructure the tree
E0505	“cannot move out of <code>*x</code> behind a <code>&</code> ”		

reference” inside the gather | you can’t move out of a shared borrow | copy the small fields (`*size_key.into_inner()`), not the whole box |

E0596	calling a mutating method through <code>&self</code>	“cannot borrow as mutable” (receiver is <code>&self</code>)	change the method receiver
E0753	FFI panic crossed the boundary	“panic in a function that cannot unwind”	never let a panic reach the FFI

Read the first column as “the code are paths where the borrow checker caught a C++-shaped habit.” Each fix is the corresponding Rust-shaped habit.

20.2 Self-check index (the Roadmap’s questions, collected)

Answers you should be able to give — by milestone — without looking. If one is fuzzy, that’s the part to re-read before implementing.

#	Milestone	The self-check question
M0	split	who owns cost / search / thread / UI / budget?
M1	types	why Pose is Copy but a Vec-wrapping type isn’t?
M2	BTreeMap	why can’t f64 be a key; what’s the minimal fix?
M3	gather	why &self; what breaks if you clone the box out?
M4	POH	how do you know convex_hull didn’t touch the tree?
M5	trisect	why pop/remove OUT before trisecting? why no in-place map mutation?
M6	Cost	why a trait? when impl Fn vs Box<dyn Cost>?
M7	cxx	what two things can’t cross a cxx boundary?
M8	errors	Result-in vs status-out: what’s lost at the edge and why OK?
M9	assemble	walk one Hyperbox borrowing/move end-to-end.
A	borrow	identify E0505 vs E0502 vs E0499 when you see them.
B	lifetime	why CppCost isn’t auto-Send/Sync; how one thread dodges it.
C	RefCell	run the 3-step checklist before adding RefCell for the counter.
D	Option	three correct ways to get an f64 out of an Option<f64>, and when each.
E	run loop	trace calls/FFI-hops/callback order for one iteration of a 4-box hull.
F	testing	which test tier catches a tie-break divergence, and why not the others?
G	translate	std::function → what? std::variant → what? what changes, not just the name?
H	debugging	first two commands when the linker or the layout goes wrong.

Use this table as a study checklist: each row is ~10 minutes of reading in the linked chapter plus one quiz. When you can answer all eighteen, you own every concept the handshake runs on.

And the honest ordering, if you only have a week: Toolchain → exercises E1-E7 → Appendix D → Appendix A → Appendix E → Appendix F Tier 3. That path ends with a Rust DIRECT that provably matches the C++ on a recorded cost sequence, which is the only milestone that actually de-risks the port. The rest of this doc is reference you’ll re-read as the compiler sends you back to it.